

Generating Folk Melodies with a Fine-Tuned Language Model

Pragna Urs

Task and Motivation

For this project I built a generative model that composes new folk melodies. Tunes written in ABC notation are just plain text, so tune generation can be framed as a sequence modeling problem, the same setup used for text generation but applied to a music domain. I found this an interesting test case because, unlike natural language, music has an additional, checkable notion of structural validity (bar lines, key signatures, repeated phrase structure), which gives a concrete way to judge whether the model actually learned something, rather than just producing plausible-looking characters.

Dataset

I used the Nottingham Music Database, a collection of 1200 British and American folk tunes written in ABC notation, via a pre-cleaned single-file version totaling 266,864 characters. I split the tokenized data into training and held-out sets (90/10, 793 vs. 89 chunks of 256 tokens) so evaluation reflects generalization to unseen tunes rather than memorization.

Model

I initially implemented a small LSTM (embedding layer, two-layer LSTM, linear output head) trained from scratch on the character level, as a baseline. Based on feedback from my professor, I changed this by fine-tuning a pretrained GPT-2 model (124M parameters, via the HuggingFace Transformers library) on the same dataset instead of training purely from scratch. The reasoning behind fine-tuning is that GPT-2 already encodes general sequence structure learned from a very large pretraining corpus, so adapting it to a comparatively small, specialized dataset such as ABC notation should need far less training than learning sequence modeling entirely from zero. I fine-tuned GPT-2 for 6 epochs (1194 steps, about 7.5 minutes on a single T4 GPU) using the standard causal language modeling objective, with the dataset chunked into blocks of 256 tokens.

As a second improvement, I implemented constrained decoding at generation time. I built an allow-list of every GPT-2 vocabulary token whose decoded text is made up only of characters that actually occur in the training data (5,459 of the model's 50,257 tokens), and masked the score of every other token to $-\infty$ before sampling. This structurally prevents the model from generating characters that could never belong to valid ABC notation, rather than merely discouraging it statistically through training alone.

Evaluation Setup

I evaluate the model using perplexity on the held-out tunes, computed as the exponential of the cross-entropy loss. Perplexity is the standard evaluation metric for autoregressive sequence models and is directly tied to the training objective, which makes it easy to justify and lets me compare the fine-tuned model against the from-scratch LSTM baseline using the same metric on the same data split.

Results

After fine-tuning, the model reaches a held-out loss of 1.086, corresponding to a held-out perplexity of 2.96. Figure 1 shows training and held-out loss over the course of fine-tuning. Both curves decrease smoothly and converge by roughly step 900, with the held-out loss staying close to the training loss throughout, which suggests the model is not simply memorizing the training tunes.

Example output from unconstrained sampling (temperature 0.5), starting from the prompt `M: 4/4 K: G`, compared against the same prompt with constrained decoding enabled:

```
Unconstrained: d|g2g gec|B2G BAG|c3 -c2B|c3 c2B|A3 -A2B|g2g gec|...
Constrained: D|G2G B2G|A2A A2B|c2c c2c|d2d B2d|G2G B2G|...
```

Both samples are syntactically well-formed ABC notation, with valid headers, consistent bar-line structure, and the kind of repeated phrase pattern typical of real folk tunes. I did not observe a large qualitative difference between the constrained and unconstrained samples in this run, which is not entirely surprising given how low the perplexity already is: the fine-tuned model appears to have already learned to stay within valid ABC characters most of the time even without the constraint, so here constrained decoding mainly acts as a structural guarantee rather than a visible fix to broken output.

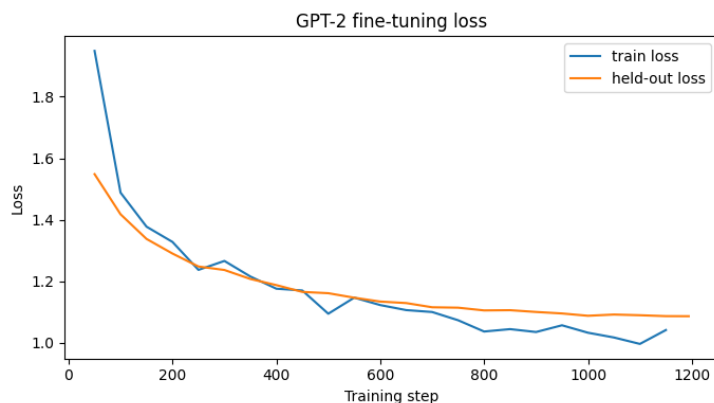


Figure 1: Training and held-out loss during GPT-2 fine-tuning.

Possible Improvements

A few directions could take this further. Constrained decoding here only enforces character-level validity, not deeper musical validity such as balanced bar durations or a consistent key signature throughout a tune; a grammar-aware decoder built around an actual ABC parser would be a meaningfully stronger constraint. It would also be informative to evaluate the from-scratch LSTM baseline on the exact same held-out split and report its perplexity side by side with the fine-tuned model, to quantify how much fine-tuning actually helped rather than assuming it did. Separately, GPT-2 was pretrained mostly on English text rather than music, so part of its advantage here may come from general sequence-modeling ability rather than anything music-specific; comparing against a model pretrained on symbolic music would help disentangle the two. Finally, scoring generated tunes with an ABC parser to measure what fraction are fully syntactically valid would give a second, complementary quantitative metric alongside perplexity.